# Classification Using Decision Trees

## 1. Aim

To understand the concept of Decision Tree classification and to implement decision tree induction using the two classical algorithms — ID3 (which splits nodes using Information Gain / Entropy) and CART (which splits nodes using the Gini Index) — in the R programming language, and to evaluate the resulting models.

## 2. Software / Packages Required

- R (version ≥ 4.0) and RStudio (recommended IDE)
- rpart package — builds classification/regression trees (supports both Gini and Information-Gain splitting)
- Base graphics (plot, text) — used to visualise the tree; no extra plotting package is required

Install rpart (only needed once): `install.packages("rpart")`

## 3. Theory

### 3.1 Decision Tree

A decision tree is a supervised learning model represented as a tree structure, where each internal node tests an attribute, each branch represents the outcome of the test, and each leaf node holds a class label. Starting at the root, an unseen record is classified by following the branch matching its attribute values until a leaf is reached.

### 3.2 ID3 Algorithm

ID3 builds a tree top-down by choosing, at each node, the attribute that produces the largest Information Gain — i.e. the attribute that most reduces the disorder (entropy) of the class labels.

Entropy of a set S with respect to the target class:

$$Entropy(S) = - \Sigma\ pi\ log2(pi)$$

Information Gain of attribute A on set S:

$$Gain(S, A) = Entropy(S) - \Sigma\ (|Sv|\ /\ |S|) \times Entropy(Sv)$$

At every node, ID3 picks the attribute with the maximum Gain(S, A) as the splitting attribute; the process repeats recursively on each child subset until all records in a node belong to one class (or no attributes remain).

### 3.3 CART Algorithm (Classification and Regression Trees)

CART builds a strictly binary tree, splitting each node using the attribute (and threshold, for numeric attributes) that minimises the Gini Index of the resulting child nodes.

Gini Index of a set S:

$$Gini(S) = 1 - \Sigma\ (pi)^2$$

A Gini value of 0 means the node is perfectly pure (all records belong to one class); CART evaluates every possible binary split and chooses the one that gives the lowest weighted Gini Index of the two children.

### 3.4 ID3 vs CART

| Feature | ID3 | CART |
|---|---|---|
| Splitting criterion | Information Gain (Entropy) | Gini Index |
| Tree structure | Can be multi-way (one branch per attribute value) | Strictly binary |
| Attribute types | Best suited to categorical attributes | Handles both numeric and categorical attributes |
| Target variable | Classification only | Classification and Regression |
| Pruning | No built-in pruning (prone to overfitting) | Uses cost-complexity pruning |

## 4. Experiment 1 — CART (Gini Index) on the Iris Dataset

### 4.1 Problem Statement

Build a CART decision tree classifier on R's built-in iris dataset (150 flowers, 4 numeric features, 3 species) to predict the Species from Sepal.Length, Sepal.Width, Petal.Length and Petal.Width. Split the data 70:30 into training and test sets, evaluate the model with a confusion matrix, and visualise the tree.

### 4.2 R Code

```
library(rpart)

set.seed(123)                      # for reproducible train/test split
data(iris)

# 1. Split data into training (70%) and testing (30%) sets
n <- nrow(iris)
train_index <- sample(1:n, size = 0.7 * n)
train_data <- iris[train_index, ]
test_data  <- iris[-train_index, ]

# 2. Build the CART decision tree (Gini index is rpart's default)
cart_model <- rpart(Species ~ Sepal.Length + Sepal.Width +
                      Petal.Length + Petal.Width,
                    data = train_data,
                    method = "class",
                    parms = list(split = "gini"))

print(cart_model)
print(cart_model$variable.importance)

# 3. Visualise the tree
plot(cart_model, uniform = TRUE, main = "CART Decision Tree - Iris Dataset")
text(cart_model, use.n = TRUE, all = TRUE, cex = 0.9)

# 4. Predict on the test set
predictions <- predict(cart_model, test_data, type = "class")

# 5. Confusion Matrix and Accuracy
conf_matrix <- table(Predicted = predictions, Actual = test_data$Species)
print(conf_matrix)
accuracy <- sum(diag(conf_matrix)) / sum(conf_matrix)
cat("Accuracy:", round(accuracy * 100, 2), "%\n")
```

### 4.3 Output

```
n= 105

node), split, n, loss, yval, (yprob)
      * denotes terminal node

1) root 105 68 virginica (0.343 0.305 0.352)
  2) Petal.Length< 2.45 36  0 setosa (1.000 0.000 0.000) *
  3) Petal.Length>=2.45 69 32 virginica (0.000 0.464 0.536)
    6) Petal.Width< 1.75 35  4 versicolor (0.000 0.886 0.114) *
    7) Petal.Width>=1.75 34  1 virginica (0.000 0.029 0.971) *
```

```
Variable importance:
Petal.Width  Petal.Length  Sepal.Length  Sepal.Width
      60.8          56.4          37.3         23.5
```

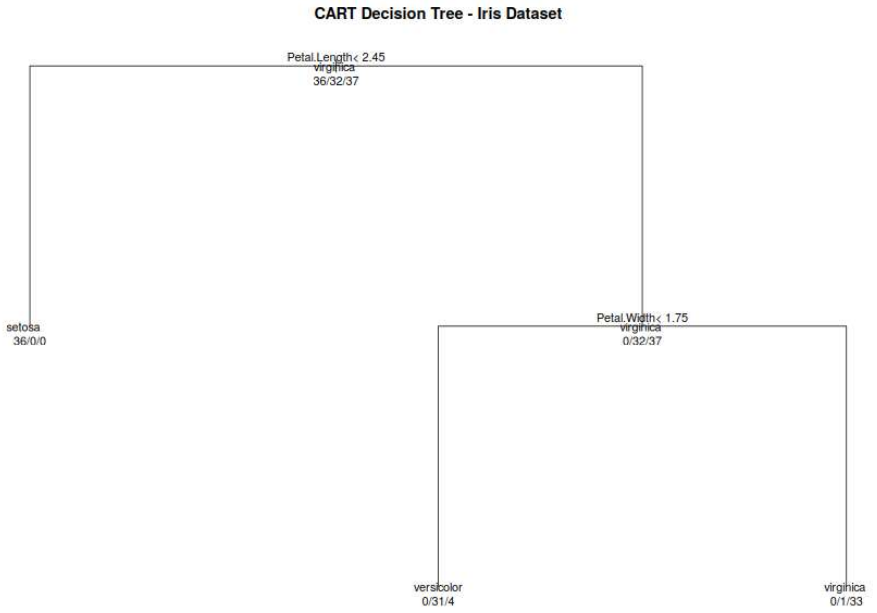



**Figure 1:** *CART decision tree for the Iris dataset. Each leaf shows the predicted class and the training-set class distribution reaching that leaf.*

## 4.4 Confusion Matrix on Test Data (45 records)

| Predicted \ Actual | setosa | versicolor | virginica |
|---|---|---|---|
| setosa | 14 | 0 | 0 |
| versicolor | 0 | 18 | 1 |
| virginica | 0 | 0 | 12 |

**Test-set accuracy = (14 + 18 + 12) / 45 = 97.78 %**

## 4.5 Interpretation

- The root split, Petal.Length < 2.45, perfectly isolates all 36 setosa flowers in one step — setosa is linearly separable from the other two species on this feature alone.
- The second split, Petal.Width < 1.75, separates most versicolor from virginica flowers; a small number of borderline flowers are misclassified, which is why the tree is not 100% accurate.
- Petal.Width and Petal.Length are the most important variables (highest variable-importance scores); Sepal measurements contribute far less.

## 5. Experiment 2 — ID3 (Information Gain) on the PlayTennis Dataset

### 5.1 Problem Statement

Using the classic 14-record PlayTennis dataset (attributes: Outlook, Temperature, Humidity, Wind; class: Play), build a decision tree using the ID3 splitting criterion (Information Gain) and use it to predict whether tennis will be played on a new day: Outlook = Sunny, Temperature = Cool, Humidity = High, Wind = Strong.

### 5.2 Dataset

| Day | Outlook | Temperature | Humidity | Wind | Play |
|---|---|---|---|---|---|
| 1 | Sunny | Hot | High | Weak | No |
| 2 | Sunny | Hot | High | Strong | No |
| 3 | Overcast | Hot | High | Weak | Yes |
| 4 | Rain | Mild | High | Weak | Yes |
| 5 | Rain | Cool | Normal | Weak | Yes |
| 6 | Rain | Cool | Normal | Strong | No |
| 7 | Overcast | Cool | Normal | Strong | Yes |
| 8 | Sunny | Mild | High | Weak | No |
| 9 | Sunny | Cool | Normal | Weak | Yes |
| 10 | Rain | Mild | Normal | Weak | Yes |
| 11 | Sunny | Mild | Normal | Strong | Yes |
| 12 | Overcast | Mild | High | Strong | Yes |
| 13 | Overcast | Hot | Normal | Weak | Yes |
| 14 | Rain | Mild | High | Strong | No |

### 5.3 Manual Calculation (Root Split)

Of the 14 examples, 9 are “Yes” and 5 are “No”, so the entropy of the full set S is:

```
Entropy(S) = −(9/14)log2(9/14) − (5/14)log2(5/14) = 0.9403
```

Computing the Information Gain of every attribute (verified in R with the entropy() helper below):

| Attribute | Information Gain |
|---|---|
| Outlook | 0.2467 ← highest, chosen as root |
| Humidity | 0.1518 |
| Wind | 0.0481 |
| Temperature | 0.0292 |

Since Outlook gives the highest Information Gain, ID3 selects it as the root split. This matches the split chosen by rpart in the code below (with parms = list(split = "information")).

### 5.4 R Code

```
library(rpart)

# 1. Create the PlayTennis dataset
play_tennis <- data.frame(
  Outlook     = c("Sunny","Sunny","Overcast","Rain","Rain","Rain","Overcast",
                  "Sunny","Sunny","Rain","Sunny","Overcast","Overcast","Rain"),
  Temperature = c("Hot","Hot","Hot","Mild","Cool","Cool","Cool",
                  "Mild","Cool","Mild","Mild","Mild","Hot","Mild"),
  Humidity    = c("High","High","High","High","Normal","Normal","Normal",
                  "High","Normal","Normal","Normal","High","Normal","High"),
  Wind        = c("Weak","Strong","Weak","Weak","Weak","Strong","Strong",
                  "Weak","Weak","Weak","Strong","Strong","Weak","Strong"),
  Play        = as.factor(c("No","No","Yes","Yes","Yes","No","Yes",
                  "No","Yes","Yes","Yes","Yes","Yes","No"))
)

# 2. Build the tree using the ID3-style criterion: Information Gain
#    (rpart exposes this via parms = list(split = "information"),
#     as an alternative to CART's default parms = list(split = "gini"))
id3_model <- rpart(Play ~ Outlook + Temperature + Humidity + Wind,
                   data = play_tennis,
                   method = "class",
                   parms = list(split = "information"),
                   control = rpart.control(minsplit = 1, minbucket = 1))

print(id3_model)

# 3. Visualise the tree
plot(id3_model, uniform = TRUE, main = "ID3-style Decision Tree - PlayTennis")
text(id3_model, use.n = TRUE, all = TRUE, cex = 0.9)

# 4. Predict for a new, unseen day
new_day <- data.frame(Outlook = "Sunny", Temperature = "Cool",
                      Humidity = "High", Wind = "Strong")
predict(id3_model, new_day, type = "class")
```

### 5.5 Output

```
n= 14

node), split, n, loss, yval, (yprob)
      * denotes terminal node

 1) root 14 5 Yes (0.357 0.643)
   2) Outlook=Rain,Sunny 10 5 No (0.500 0.500)
     4) Humidity=High 5 1 No (0.800 0.200)
       8) Outlook=Sunny 3 0 No (1.000 0.000) *
       9) Outlook=Rain 2 1 No (0.500 0.500)  ... (further splits)
     5) Humidity=Normal 5 1 Yes (0.200 0.800)  ... (further splits)
   3) Outlook=Overcast 4 0 Yes (0.000 1.000) *

Prediction for (Sunny, Cool, High, Strong): Play = No
```

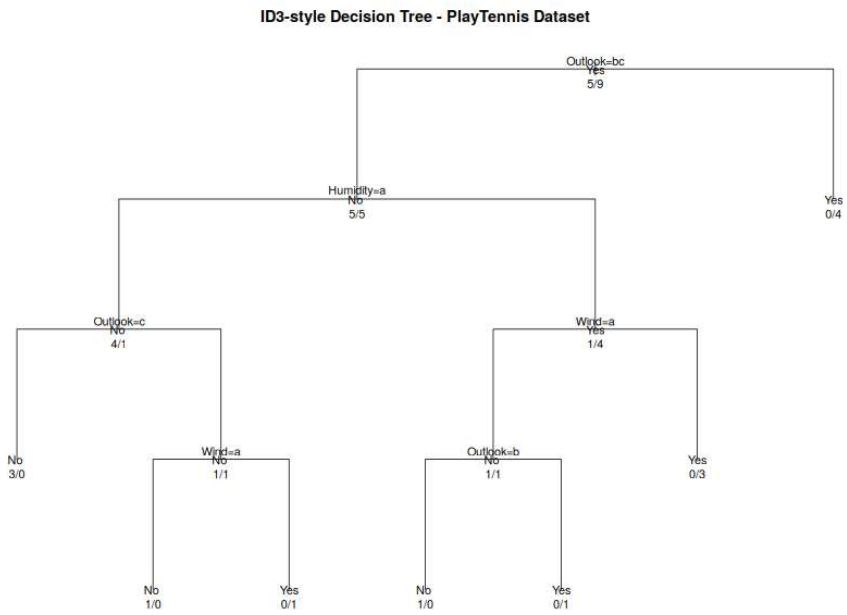



**Figure 2:** *ID3-style tree for PlayTennis. Base R's plot.rpart() codes categorical levels alphabetically as a, b, c … — see the legend below to read the split labels.*

**Reading the plot:** plot.rpart() prints categorical splits using letter codes in alphabetical order of the factor levels: Outlook → a=Overcast, b=Rain, c=Sunny; Humidity → a=High, b=Normal; Wind → a=Strong, b=Weak. For example "Outlook=bc" at the root means {Rain, Sunny} — matching the printed tree text above. The printed model summary (Section 5.5) always shows the real category names and should be treated as the primary reference.

### 5.6 Training Accuracy

| Predicted \ Actual | No | Yes |
|---|---|---|
| No | 5 | 0 |
| Yes | 0 | 9 |

**Training accuracy = 14/14 = 100 %** (expected on such a small dataset with no pruning — the tree memorises the training data perfectly, which is also a classic illustration of ID3's tendency to overfit).

## 6. Result

Decision tree classifiers were successfully built using both the CART (Gini Index) and ID3 (Information Gain) splitting strategies in R using the rpart package:

- CART on the Iris dataset achieved 97.78% accuracy on unseen test data, with Petal.Length and Petal.Width identified as the most discriminating features.
- ID3 on the PlayTennis dataset correctly identified Outlook as the best root attribute (Information Gain = 0.2467), matching the hand-calculated entropy values, and correctly predicted "No" for the unseen test day.

Both experiments confirm that decision trees provide an interpretable, rule-based classification model whose structure can be read directly as a set of if-then decision rules.